



Editorial- Prime Maze

Problem Setter:

Hok Fong Wong (SWong)
Hou Kong Middle School

2022/06/04 @ PCOI



Problem Statement

PDF Link:

https://drive.google.com/file/d/1punGJKb_BGUvtl7WBwMiRUJCKd4ZY6cn/view?usp=sharing

- ▶ Navigate a $R \times C$ maze with prime numbers being the obstacles.
- ▶ You can use a **move** to travel to adjacent blocks.
- ▶ Calculate the smallest number of move(s) required to travel from $(1, 1)$ to (R, C) .

- ▶ **Multiple test cases** ($T \leq 10^6$).
- ▶ **Constraints:** $R \leq 2^{31}-1, C \leq 20$.

R is huge, C is very small...

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40

Subtasks (Invisible)

- ▶ **Subtask 1** (10 pts): $R \leq 5, C \leq 5$
- ▶ **Subtask 2** (20 pts): $T \leq 400, R \leq 20$
- ▶ **Subtask 3** (10 pts): $T \leq 1000, R \leq 1000$
- ▶ **Subtask 4** (60 pts): No further constraints

Statistics

- ▶ **Max.** 30 pts by **R8G1(PCMS)**
- ▶ **Min.** 10 pts
- ▶ Expected score: 40 pts by **exhaustion**

Subtask 1

- ▶ Idea: How to get some score if you don't even understand the problem??
- ▶ Output `-1` !
- ▶ ☹️☹️ However, the test data includes a special case where $R = 1, C = 1$, and the program should answer this kind of query with `0` instead.



Subtask 2&3

- ▶ Hey, we can use **breadth-first search**!
- ▶ Time complexity: $O(TRC)$
- ▶ **Not depth-first search!** We are looking for the smallest moves.

- ▶ Distinction between **DFS** and **BFS**:

DFS is usually for obtaining every possible paths by trial-and-error.

BFS is aimed expanding the states one at a time, so that the states using smaller moves will always be visited once and always the first.

Subtask 2&3

Procedures:

1. Use Prime Sieve to obtain all prime numbers within the $R \times C$ range
2. Construct the maze explicitly
3. Apply the BFS algorithm

Don't forget to memset/clear your arrays 😊

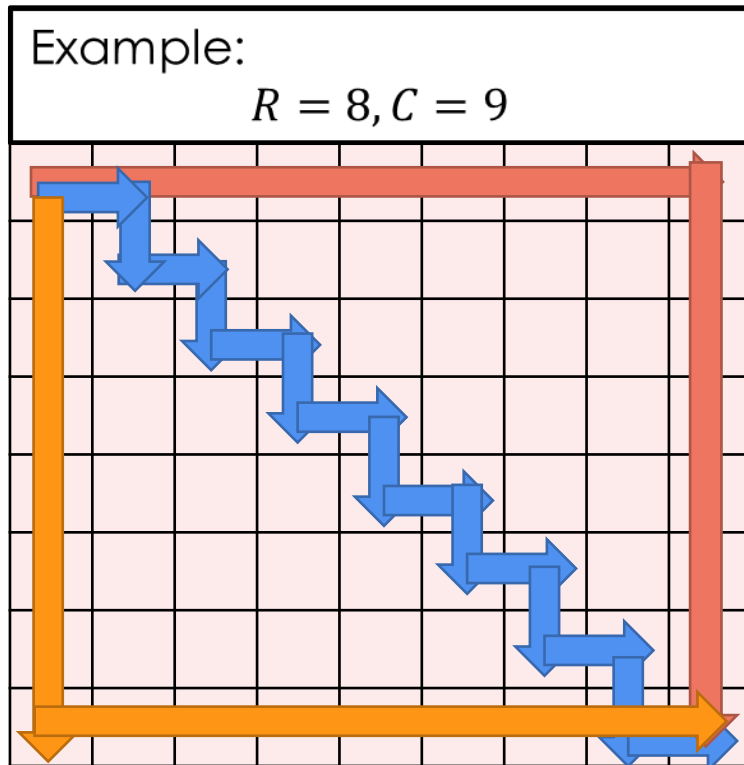
Subtask 4

Strategies required:

- ▶ Trial-and-error
- ▶ Observations and proving
- ▶ Classification
- ▶ Combining the subtasks together

Subtask 4

- ▶ Solving Subtask 2&3, you now have a nice and clear solution for small test cases!
- ▶ Consider a $R \times C$ maze without any obstacles:



Smallest moves:

$$|8 - 1| + |9 - 1| = 15$$

Paths colored **red**, **blue** and **orange** are some possible paths with smallest moves.

Subtask 4

- ▶ What will cause the answer to increase?
- ▶ Hints already provided in the statement!

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40

What a waste!
step+=2

Subtask 4

Key Observation 1

The **last block in a row (except the first)** is always *accessible*.

Proof. The number in block (r, C) is $(r - 1) \times C + C = r \times C$ (where $r > 1$), implying that this block is always accessible as it contains a composite number.

- ▶ Hence, we just need to find a way to connect the two columns together.
- ▶ Given C , the numbers that appear on the C^{th} column are “fixed”.
- ▶ Increasing R can be seen as revealing the numbers in a row-by-row fashion.

Case #1: $C=1$

- Let's just print the maze out and observe!

First 5 lines:

Example:

$$R = 5, C = 1$$

1
2
3
4
5

Conclusion:

Solvable if and only if $R = 1$

Else output "-1"

Case #2: "I'm stuck!"

Key Observation 2

No path exists as the maze is completely blocked by obstacles when $C = 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 15, 16, 18$.

Example 1:
 $R = 5, C = 2$

1	2
3	4
5	6
7	8
9	10

Example 2:
 $R = 5, C = 3$

1	2	3
4	5	6
7	8	9
10	11	12
13	14	15

Example 3:
 $R = 5, C = 7$

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	32	33	34	35

Conclusion:
Output **"-1"**

Case #3: Let's find the “winning strategy”

Key Observation 3

When $C = 8, 14, 17, 19, 20$, we can always find a path connecting the first and the last column without “wasting” any moves, provided that $R \geq R_0(C)$, where $R_0(C)$ is a small constant that varies depending on C .

Example 1:
 $R_0 = 7, C = 8$

Case: $r=7, c=8$

01101010
00101000
10100010
00001010
00001000
10100010
00001000

Strategy:

01101010
00101000
10100010
00001010
00001000
10100010
00001000

Other value of R_0 s:

For $C = 14, R_0 = 8$

For $C = 17, R_0 = 11$

For $C = 19, R_0 = 10$

For $C = 20, R_0 = 11$

Conclusion:

When $R \geq R_0$, print the taxicab distance between $(1, 1)$ and (R, C)

Case #4: Cases left when R is small

Key Observation 4

When $C = 8, 14, 17, 19, 20$, and $R < R_0(C)$, the answer may be larger than $R + C - 1$.

- ▶ Generic and small enough, try using **BFS**!
- ▶ Use a **table** to avoid repetitive computations!

Conclusion: 😊

Combine the solutions together!

Possible Errors and Solutions

Error	Expected score	Solution
1. Missing the case where $R = 1, C = 1$	80 (WA on test case 1,3)	Output 0 when $R = 1, C = 1$
2. Using BFS every time solving subtask 4 when $R < 20$	90 (TLE)	Save the table beforehand and do $O(1)$ queries
3. Not using <code>long long</code> ($R + C - 2$ could be exceeding <code>int</code>)	90	Change variable declarations to <code>long long</code>